

Student Marks Analyzer

A Functional Programming Approach

Group No - 21

Group Members:

EG/2020/4212 - Sewwandi L.L.C

EG/2020/4214 - Shamali A.V

EG/2020/4268 - Weerasinghe W.M.N.S

EG/2020/4277 - Welikumbura R.W.R.L.

Git Repo link : <https://github.com/Nishadi4268/student-marks-analyzer-Functional-Programming>

Explanation Video Link : <https://youtu.be/C9BrIBiwy8I>

1. Problem Statement and Industrial Motivation

1.1 Problem Statement

Educational institutions process thousands of student records requiring statistical analysis, grade computation, and performance reports. Traditional imperative solutions suffer from mutable state bugs, concurrency issues, and testing difficulties.

1.2 Industrial Motivation

Modern educational data systems face several challenges:

1. **Data Integrity** : Immutability ensures records remain unchanged during analysis
2. **Scalability** : Pure functions enable safe parallelization for processing millions of records
3. **Audit Trail** : Reproducible calculations for academic compliance
4. **Regulatory Compliance** : FERPA/GDPR transparency through functional pipelines

2. Functional Design

2.1 Core Data Types (DataTypes.hs)

```
data Student = Student
  { studentId :: String
  , name      :: String
  , marks     :: [Double]
  } deriving (Show, Eq)
data Grade = A | B | C | D | F deriving (Show, Eq, Ord)
data Statistics = Statistics
  { average :: Double
  , median  :: Double
  , stdDev  :: Double
  , maxScore :: Double
  , minScore :: Double
  } deriving (Show)
data AnalysisReport = AnalysisReport
  { statistics      :: Statistics
  , gradeDistribution :: [(Grade, Int)]
  , topPerformers   :: [Student]
  , atRiskStudents  :: [Student]
  } deriving (Show)
```

2.2 Main Functions

```
-- Processing.hs
calculateAverage :: [Double] -> Double
calculateMedian  :: [Double] -> Double
computeGrade    :: Double -> Grade
analyzeStudents :: [Student] -> AnalysisReport

-- IOHandler.hs
readStudentData :: FilePath -> IO [Student]
writeReport     :: FilePath -> AnalysisReport -> IO ()

-- Utils.hs
quickSort :: Ord a => [a] -> [a]
percentage :: Int -> Int -> Double
```

2.3 Program Flow (Main.hs)

```
main :: IO ()
main = do
  students <- readStudentData "sample_data.csv"      -- I/O
  let report = analyzeStudents students              -- Pure computation
  putStrLn $ formatReport report                    -- I/O
  writeReport "analysis_report.txt" report           -- I/O
```

3. Functional Programming Concepts Implementation

3.1 Pure Functions

Concept: Functions with no side effects that always return the same output for the same input.

```
calculateAverage :: [Double] -> Double
calculateAverage [] = 0
calculateAverage xs = sum xs / fromIntegral (length xs)
```

Benefits : Testable, parallelizable, cacheable.

3.2 Algebraic Data Types (ADT)

Concept: Type-safe domain modeling with sum and product types.

```
data Grade = A | B | C | D | F -- Sum type
computeGrade :: Double -> Grade
computeGrade score
  | score >= 90 = A
  | score >= 80 = B
  | score >= 70 = C
  | score >= 60 = D
  | otherwise  = F
```

Benefits: Impossible states unrepresentable, compiler-enforced exhaustive matching.

3.3 Recursion

Concept : Functions calling themselves instead of using loops.

```
quickSort :: Ord a => [a] -> [a]
quickSort [] = []
quickSort (pivot: rest) =
  quickSort smaller ++ [pivot] ++ quickSort larger
  where
    smaller = [x | x <- rest, x <= pivot]
    larger  = [x | x <- rest, x > pivot]
```

Benefits: Natural for list processing, easier to prove correctness.

3.4 Higher-Order Functions

Concept : Functions that take functions as arguments or return functions.

```
getTopPerformers :: [Student] -> [Student]
getTopPerformers = filter (\s -> calculateAverage (marks s) >= 90)
```

```
assignAllGrades :: [Student] -> [(Student, Grade)]
assignAllGrades = map (\s -> (s, computeGrade $ calculateAverage $ marks s))
```

Benefits: Code reuse, declarative style, built-in parallelization.

3.5 Immutability

Concept : Data cannot be modified after creation.

```
updateStudentMarks :: Student -> [Double] -> Student
updateStudentMarks student newMarks =
    student { marks = newMarks } -- Creates NEW student
```

Benefits : Thread-safe by default, no defensive copying.

3.6 Function Composition

Concept : Combine simple functions to build complex operations.

```
analyzeStudents :: [Student] -> AnalysisReport
analyzeStudents =
    generateReport . calculateStats . assignGrades . validateData
```

Benefits : Readable pipelines, independently testable stages.

3.7 Lazy Evaluation

Concept: Expressions evaluated only when needed.

```
processLargeDataset :: [Student] -> [AnalysisReport]
processLargeDataset = map analyzeStudent -- Computed lazily
```

Benefits : Memory efficient, enables infinite data structures.

4. Expected Outputs and Validation

Sample Input: 5 students with 3 subject scores each

Console Output:

```
===== Student Marks Analyzer Report =====
Number of students: 5
Cohort average: 69.1
Cohort average: 69.1
Cohort average: 69.1

Top 5 students:
- Rala Fernando (5003): 91.7
Cohort average: 69.1

Top 5 students:
Cohort average: 69.1
Cohort average: 69.1
Cohort average: 69.1

Top 5 students:
- Rala Fernando (5003): 91.7
- Ayesha Perera (5001): 73.7
- Himali Perera (5005): 67.3
- Rasan Silva (5002): 64.3
- Kasun Jay (5004): 48.3

Grade distribution:
- A: 1
- B: 1
- C: 2
- D: 0
- F: 1

Per-student summaries:
Ayesha Perera (5001) -> avg: 73.7, max: 91, min: 48, grade: B
Rasan Silva (5002) -> avg: 64.3, max: 70, min: 59, grade: C
Rala Fernando (5003) -> avg: 91.7, max: 95, min: 89, grade: A
Kasun Jay (5004) -> avg: 48.3, max: 52, min: 45, grade: F
Himali Perera (5005) -> avg: 67.3, max: 76, min: 54, grade: C
=====
```

5. Possible Extensions

1. **Parallel Processing** : Use `Control.Parallel` for concurrent analysis
3. **Advanced Analytics** : Correlation analysis, trend prediction
4. **Visualization** : Generate charts using Chart library
5. **Database Integration** : Store results in PostgreSQL

6. Why FP Improves Reliability and Concurrency

6.1 Reliability

Type Safety : Compiler catches errors before runtime

```
badGrade :: Grade
badGrade = 95 -- Compiler error! Type mismatch caught before runtime
```

Purity Guarantees: Functions cannot modify global state, same input always produces same output, no hidden side effects.

Immutability: Original data never corrupted, no accidental modifications.

6.2 Concurrency

No Race Conditions: Safe parallel execution without locks

```
thread1 = analyzeStudents group1
thread2 = analyzeStudents group2 -- No locks needed!
```

Easy Parallelization:

```
import Control.Parallel.Strategies
analyzeParallel students =
  map analyzeStudent students `using` parList rdeepseq
```

Deterministic Results: Sequential and parallel execution produce identical results.

6.3 Comparison

Aspect	Imperative	Functional
State	Mutable, hard to track	Immutable, easy to reason
Concurrency	Locks required	Lock-free parallelization
Testing	Complex mocking	Simple pure function tests
Bugs	Non-deterministic	Reproducible errors

7. Conclusion

This project demonstrates that functional programming provides:

- **Correctness** - through type safety and purity
- **Reliability** - via immutability and referential transparency
- **Concurrency** - without locks or race conditions
- **Maintainability** - through modular, composable functions

Real-world companies (Facebook, Jane Street, Target) use FP for these exact benefits. This Student Marks Analyzer proves FP is practical for building robust, production-ready systems